

基于2D栅格地图的路径规划与导航，动态调整与纠偏

一、任务描述

上一节我们已经了解，自动驾驶汽车实现自动驾驶需要解决的三个主要问题是“我在哪？”，“我要去哪？”，“如何去？”。ROS建图功能能够解决自动驾驶汽车的定位问题，至于如何从起点行驶到终点，就涉及到路径规划的问题，路径规划可以分为全局路径规划和动态路径规划（也可以叫局部路径规划）：

全局路径规划：全局路径规划算法的目标是在给定障碍物地图、起始点和终点的情况下，找到一条连接起始点和终点且针对障碍物无碰撞的路径。

动态路径规划：动态是指在沿着最优路径前进时，突然出现了既定路径中断的情况需要进行重新规划。

本次实训任务是操作自动驾驶汽车，在地图的起点和终点已知的情况下，通过全局路径规划和动态路径规划算法，让自动驾驶汽车从起点行驶到终点，自动躲避障碍物。

二、任务目标

- 了解自动驾驶路径规划常用算法和算法原理。
- 掌握自动驾驶汽车基本操作。
- 能够使用自动驾驶汽车实现全局路径规划和动态路径规划。

三、任务环境

- 操作系统：Ubuntu20.04 ROS-neotic
- 工具软件：NoMachine
- 必备硬件：自动驾驶汽车

四、任务原理

4.1 全局路径规划

现有的全局路径规划算法种类繁多，有基于图搜索的dijkstra算法和a star算法等；还有基于采样的路径规划算法，如概率路线图(probabilistic roadmaps，简称prm)算法、prm star算法(prm算法的变种)、快速扩展随机树(rapidly-exploring randomtrees，简称rrt)算法和rrt*算法(rrt算法的变种)等；还有基于启发式算法的路径规划算法如蚁群算法和遗传算法等。以下仅仅针对一些被广泛使用的全局路径规划算法进行展开。

4.1.1 Dijkstra算法

Dijkstra算法是由荷兰计算机科学家狄克斯特拉1959年提出的，因此又叫狄克斯特拉算法。是从一个顶点到其余各顶点的最短路径算法，解决的是最短路径问题。Dijkstra算法主要特点是以起始点为中心向外层层扩展，直到扩展到终点为止。

这一算法可以用于解决在地图已知情况下的自动驾驶汽车路径规划问题，例如：无人仓储中的自动驾驶汽车路径规划问题、餐厅中上菜机器人路径规划问题等。

前面我们已经学习了，利用栅格法，将实际的运动场景加工处理成便于计算机处理的形式，接下来我们用栅格地图讲解Dijkstra算法的工作原理，下图为栅格化的地图，其中深颜色的为障碍物，白色的为可通行区域：

A	B		
C	D		
	E	F	
	G	H	I

现在我们需要自动驾驶汽车规划从A地点到B-I这8个地点的路径，我们通过Dijkstra算法完成这个任务。A-I共9个可达到的节点（右上角节点是不可到达的点，在算法运行过程中不会添加入列表中），我们定义A为起始地点，以A为中心判断其周围的可达节点：

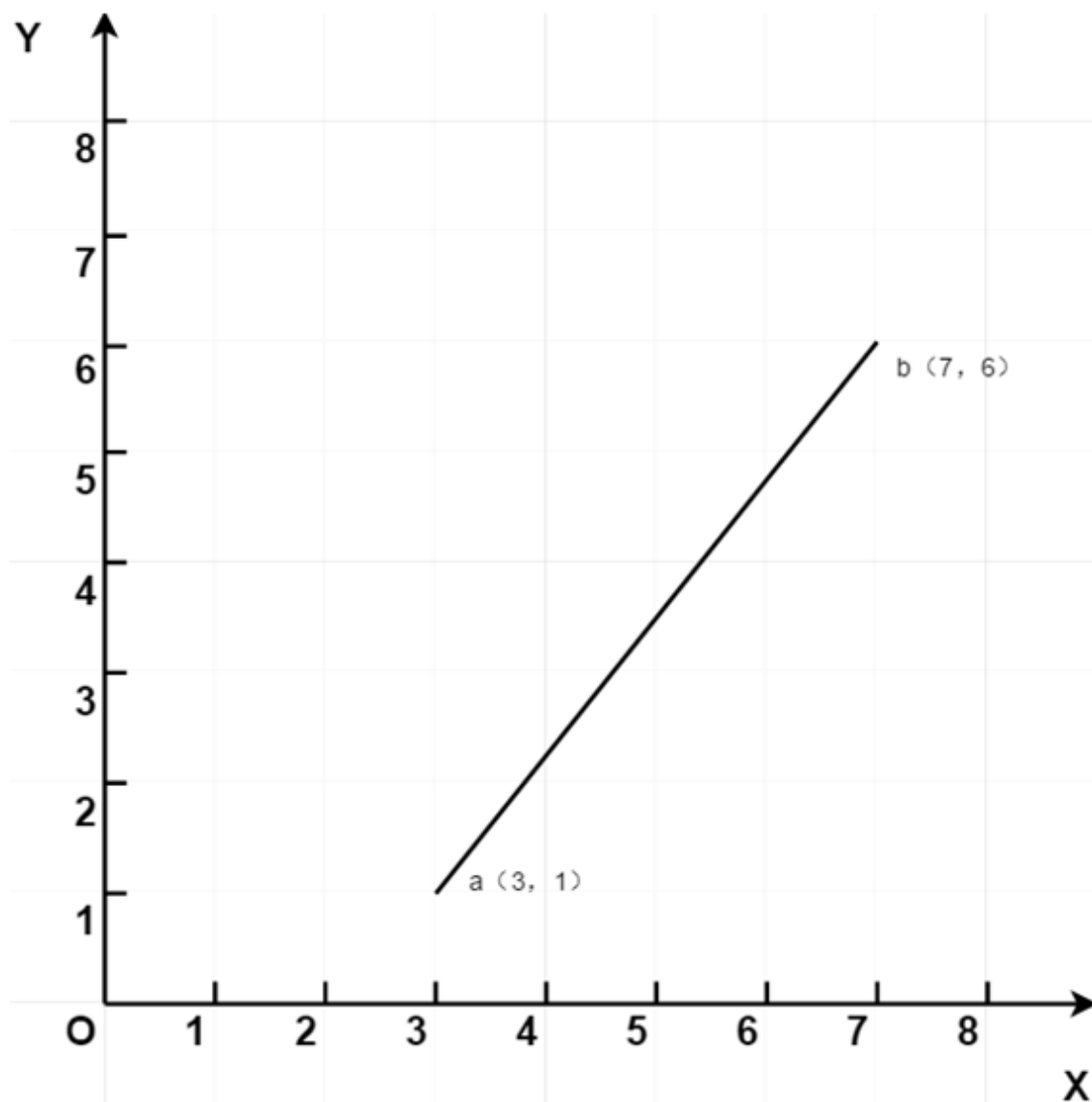
null	null	null
null	A	B
null	C	D

我们可以看到周围的八个节点中，只有B、C、D是可达到的，其余位置都是不存在的（或者说超出图片范围的），然后我们需要对B、C、D三个节点进行进一步的判断，从A到达B、C是可以直接到达的，并且B、C没有被记录（现在只在集合中记录了起始节点A），那么B、C将会被加入到下一次筛选的列表中；从A到D是需要斜着走的，这就要求斜向的路径上不能有阻碍，即B、C不能是障碍物，显然，现在A是可以直接到达D的，那么D也会被记录在下次筛选的备用列表中。

在这里介绍一个关于距离的概念：欧式距离。两个点之间的欧式距离即两点间的直线距离：

例：二维平面两点a(3,1)与b(7,6)间的欧式距离：

$$\rho = \sqrt{(7-3)^2 + (6-1)^2} = \sqrt{41}$$



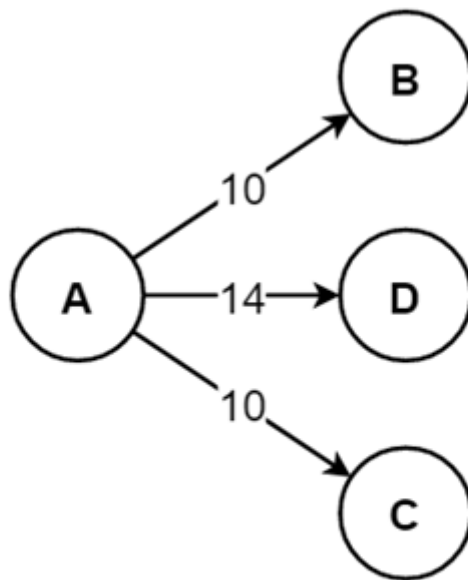
我们在图4中可以直观地看到从A出发到B、C的距离和到达D的距离是不同的，可以大致估算为10和14 ($10 \times \sqrt{2}$)，这也是需要记录的数据，对于这些要记录的数据，我们分别创建不同的储存空间来记录：

闭集 α ：记录已经确定从起点到达路径的点；

开集 β ：记录已经查找到可到达，但是没有确定路径的点；

在算法开始时，我们就把起始节点A加入到开集 β 中，在上面的例子中，第一步会把A移动到闭集 α 中，然后对A的周围节点开始检测，然后我们就可以把B、C、D加入到开集 β 中，这时会出现一个问题，我们怎么记录路径呢？

我们简单观察可以发现，从A检测到并添加到开集 β 的节点有一个共同的特点：从A出发。所以我们可以从B、C、D上记录前一步时A，和节点A的距离分别是10、14、10，假设我们并不是在一个4×4的栅格图中规划路径，而是在一个2×2的栅格图中的话，我们只需要将B、C、D加入到闭集 α ，任务就已经完成了，我们可以从每个单元格记录的数据得出，从A直接走到B、C、D就是从A出发的最短路径。



A 节点有效邻域节点距离。

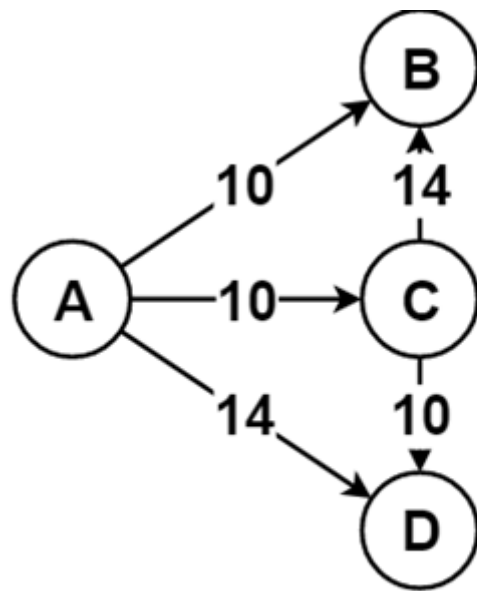
然而我们实际面临的情况是比 2×2 栅格图复杂很多的，在超过 2×2 的栅格图中，我们下一步怎么做呢？我们会在开集 β 中寻找和A距离最近的一个节点，我们只需要比较每个节点记录的数字（B、C是10，D是14），就可以得出B、C离A比较近，然后从中选择一个，将其从开集 β 移动到闭集 α ，然后检测它周围的节点，我们选择移动C节点（实际程序中选择移动B或者C可以自行调整），

那么从C检测会得出什么情况呢？

null	A	B
null	C	D
null		E

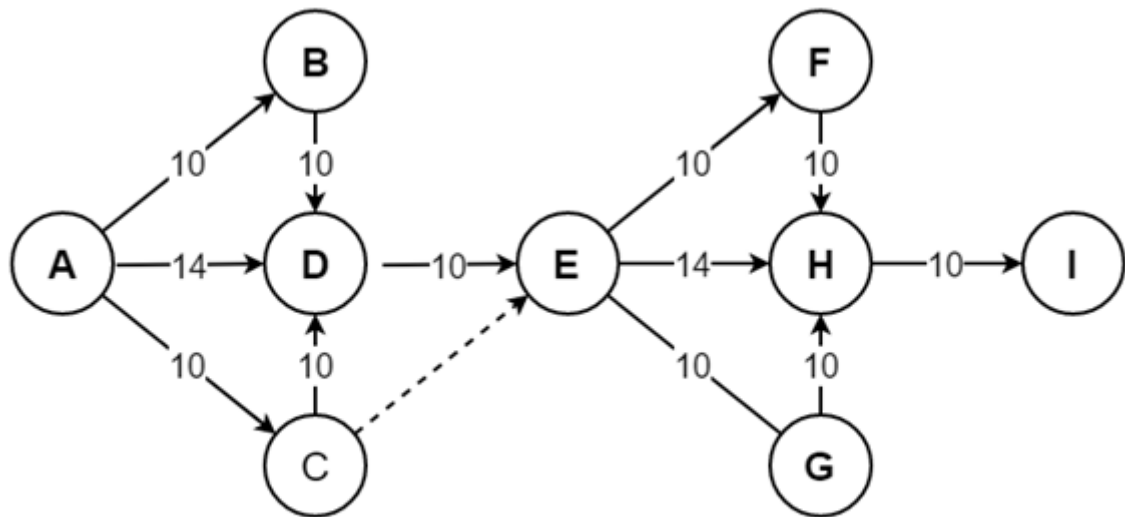
C 节点邻域分析。

我们可以看到，C左侧的三个点是不存在的，我们不需要考虑，而C相邻的A、B、D点已经可以在闭集 α 、开集 β 中找到，也不需要我们进行添加，下侧的点是一个障碍物，也不必添加，E点看起来是相邻的一个可到达节点，但是事实上呢？从C到达E并不能直接到达，C节点下侧的障碍物会阻碍从C直接到达E的路径，所以E节点也是不可达的，对于闭集 α 、开集 β ，我们只是做了将C从开集 β 移动到闭集 α ，不需要其余的操作，但是对于B、D节点呢？我们可以比较从C到达B、D的路径距离，可以得出从A路过C到达B、D的距离分别是24、20，是大于现在记录的距离的，不会采用这种消耗更大的方式。假设得出的是优于现在记录的路径，则会修改该点记录的上一步节点和距离数值，例：B（C，24）。



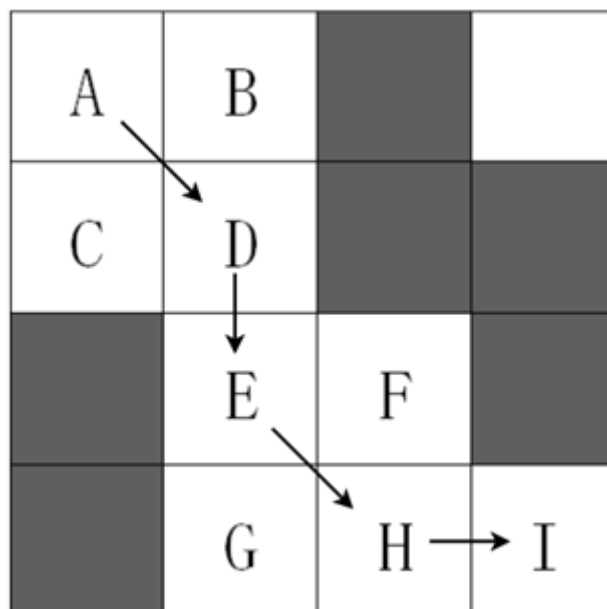
C 节点有效邻域节点距离

重复上述步骤，我们可以得出从A到每个可到达点之间的每一步距离，如图（C不可到达E）：



A 节点到达 I 节点距离分析

从而得出从A出发到达每个可到达点的距离，路径可以利用每个点记录的上一步节点名通过一步一步回溯得出，从I开始回溯的步骤：I的上一步是H，H的上一步是E，E的上一步是D，D的上一步是A，最后可得出从A到I的最优路径如下图所示：



4.1.2 A*算法

Astar算法是一种静态路网中求解合适路径最有效的启发式搜索算法，也是解决许多搜索问题的有效算法。和Dijkstra算法相比，Astar 算法的查找效率更高，能够更快地找到从起始点到达终点的路径，Astar算法得出的路径和Dijkstra算法得出的路径是完全相同的。

相比于Dijkstra算法的每走一步，观察周围，试探所有可行路径的方式，Astar算法增加了一步——利用代价函数确定搜索方向，明显缩减了节点查询的运算量。

在Astar中我们引入代价函数，来判断下一步需要到达的哪个节点，我们根据如下代价函数：

$$F = G + H$$

其中G：表示从起始节点到达当前节点的路径代价

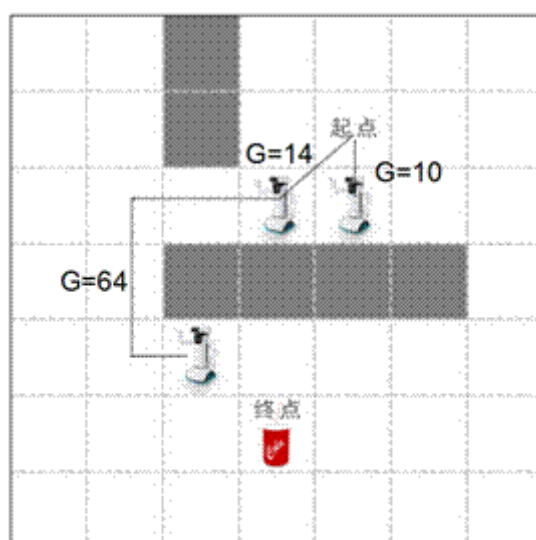


图3 G值的计算方法

H：从当前点到达终点的曼哈顿距离

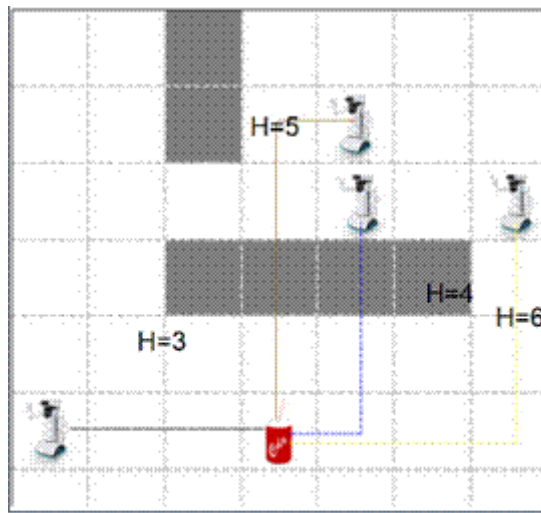


图4 H值的计算方法

计算G与H我们分别使用欧氏距离与曼哈顿距离进行计算，下面介绍新的代价函数计算方式：曼哈顿距离。

二维平面两点 $a(x_1, y_1)$ 与 $b(x_2, y_2)$ 间的曼哈顿距离：

$$D_{12} = |x_2 - x_1| + |y_2 - y_1|$$

例：二维平面两点 $a(3, 1)$ 与 $b(7, 6)$ 间的曼哈顿距离：

$$D_{ab} = |3 - 7| + |1 - 6| = 9;$$

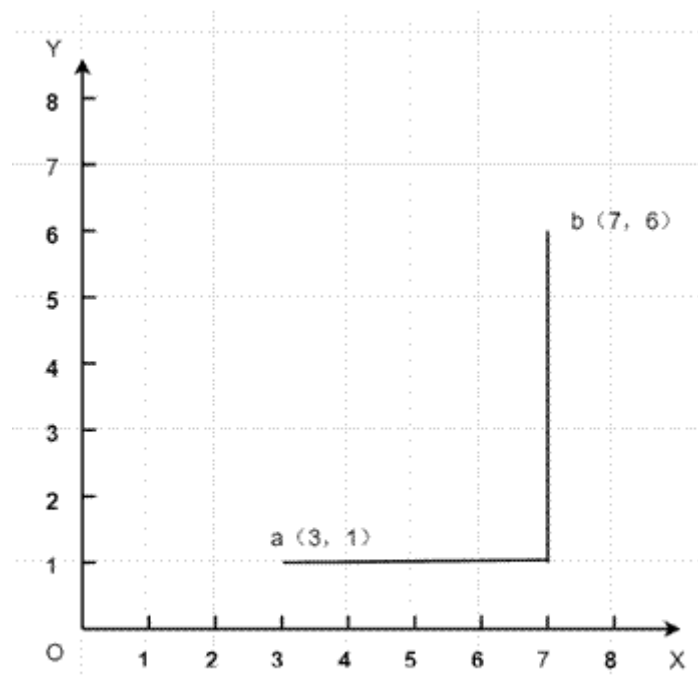


图5 曼哈顿距离示例

由于终点的位置是已知的，所以每次规划下一步路径时，本次节点周围的八个节点与终点的曼哈顿距离是可知的（不计后续路径中可能出现的障碍物），所以在路径规划过程中，不必尝试每个节点（Dijkstra算法的方式），只要向终点方向（与终点距离最短的节点）走下一步即可，相比于Dijkstra算法无方向性的层层扩展方式，显然Astar向目标前进的方式效率更高。

在算法的实现过程中，我们依旧会建立两个集合来储存查找过程中得到的信息：

闭集 α ：记录已经确认与起始点最短路径的点；

开集 β ：记录已经找到但是未确认与起始点最短路径的点；

我们将起始点到当前点的路径距离记录为G，每一步的距离使用欧氏距离进行计算，当前点到达终点的曼哈顿距离记录为H，G与H的和记录为F。

在算法开始时，在开集 β 中添加起始节点A；

首先我们会查找开集 β 中F最小的点（此时开集 β 中只有起始点A），在找到这个点后，我们将其移动到闭集 α 中，此时以A为中心，对其周围的八个节点进行检测。首先需要确定判断每个节点是否可达，可达的节点需要满足以下条件：

1. 不是障碍物
2. 不在闭集 α 中
3. 未超出地图
4. 直接可达（这一条件主要针对斜向的节点，需要斜向对应的方向相邻两节点都是可通过的，例如A—>D可行，D—>F不可行）；

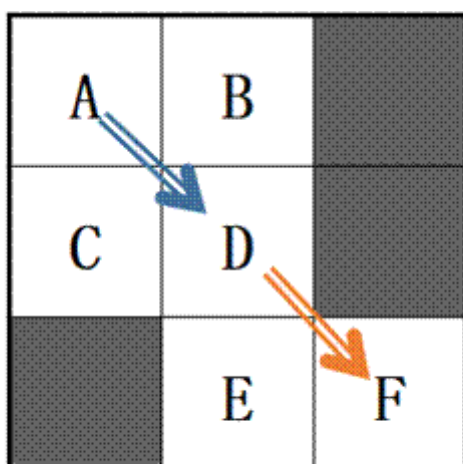


图6 A—>D无阻碍，D—>F有阻碍

完成上述判断后，对可达的点（即有效邻域节点）添加到开集 β 中，并对这些有效节点的G值、H值和F值进行计算；如果该点已经在开集中，则比较已经记录的G值和途径当前节点到达该点的G值，将其父节点记录更改为G值较小的路径（相等不修改）。

重复上述步骤直至终点出现在开集 β 中，从终点依次按照记录的父节点回溯得出规划的路径；如果直至开集 β 为空都没有找到终点，则返回空，或提示未找到路径。

在算法运行过程中，需要注意处理两个问题：

1. 开集中存在一个以上相同最小F值的节点，如何选取下一步节点。

在新的节点加入到开集 β 时，我们会利用C++的指针功能，设定一个指针始终指向开集 β 中F值最小的节点，当加入一个新的节点时，我们会将其F值与当前指针指向的节点的F值相比较，如果当前指向的节点F值较小，则不做改变；如果新加入节点的F值较小，则指向新加入的节点；如果新加入节点的F值与当前指向的节点相等，我们可以自行选择不做改变或者指向新的节点，下面例程中选择指向新加入的节点。

2. 如何利用G值更新路径

在每次检测当前节点为中心的八个节点时，如果节点可达，可以得出从起始节点到达节点的距离：起始节点到当前中心节点的距离和中心节点到达当前点的距离之和。如果该节点不在开集 β 中，则记录本次得到的G值并将其加入到开集 β ；如果节点已经在开集 β 中，表示之前已经找到另外一条到达这个节点的路径，此时开集 β 中会记录另外路径的G值，与当前得到的G值相比较，G值较小的是较短的路径，选择将节点的父节点记录为G值较小的路径的上一步节点，更新信息即可。

我们使用4×4栅格图来模拟算法具体步骤：

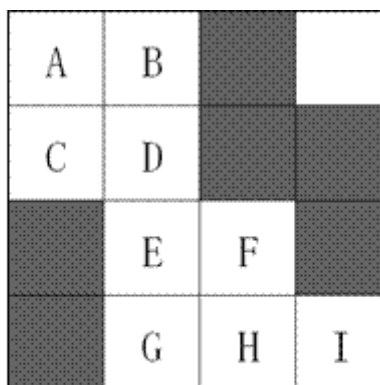


图7 Astar 原理实验例图

在图7中，我们设置起点为A，终点为I，从起始点A出发，查找到附近有效邻域节点B、C、D后，我们分别从B、C、D处向终点I的方向“看”一下，得到H值（此处采用欧式距离示例），并且将H值和G值相加，可以得到结果：B、C的F值为： $10 + \sqrt{20^2 + 30^2} = 10 + 10\sqrt{13} \approx 46$ ，而D的F值为：

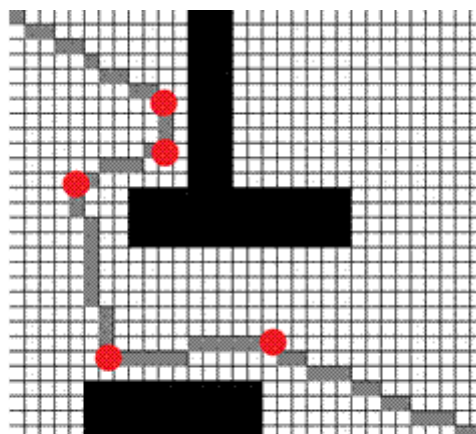
$\sqrt{30^2 + 30^2} = 30\sqrt{2} \approx 42$ ，比较可以发现从A出发途径D到达I的总距离是比较短的，在这种情况下，下一步的起始节点就会是D，以D为中心我们可以发现新的有效节点E，我们从A到E的查找只经历了两次A→D、D→E，而使用Dijkstra算法，我们需要查找判断A→B、A→C、A→D、C→B、C→D、B→D、D→E七次，显然A*算法是更高效的。

相比于Dijkstra算法，Astar更加充分的利用了全部地图已知的信息，在Dijkstra算法“视野”局限于当前节点周围的情况下，寻找终点的任务执行过程表现为从起始点开始，获取起始点周围的所有信息并全部尝试，直到“遇到”终点；而Astar算法则“视野”变得更加开阔，每走一步都会“望一望”，每次“寻找”都能趋向是离终点更近的，是一种“找到”终点的方式。

4.1.3 RRT算法

快速扩展随机树（Rapidly-exploring Random Trees-----RRT）算法由美国爱荷华州立大学的Steven M.LaValle教授在1998年提出的，是目前广泛应用的基于采样的运动规划算法。

RRT算法的思想是创建一个从起始点随机生长的树，树的生长方向是会趋于空白区域的，在障碍物较少的情况下，使用RRT可以很快的探索空间，找到通往目标节点的路径，但是这个路径不会是最优的。图中的（b）（c）可以看出，RRT规划的路径是存在折返的情况的，这是由于每一步都是随机生成，以图3举例，其节点分布如下图：

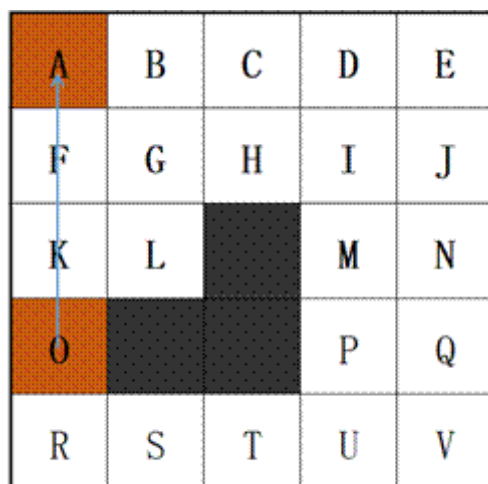


RRT 规划路径节点位置

RRT在运行前也会读取地图，但是RRT对地图的处理和之前讲过的Dijkstra算法与AStar算法不同，RRT会首先遍历每一个栅格，将所有非障碍物的栅格记录在一个新的列表中（Dijkstra和AStar是建立对应的二维数组记录每个栅格是否障碍物），此列表下文称为备选节点列表。

算法开始输入起始节点和终点后，通过随机的方式，在备选节点列表选取一个点，然后在随机树上找一个与其距离最短的点（第一次运行时随机树只有起始节点这个根节点存在），然后尝试连接两点，如果连接两点的直线上没有障碍物，就可以认为当前选中的点是可以作为树上的节点的，此时将选中节点的父节点记录为树上距离它最近的点，对应到树，可以认为这一次的“枝”是由“树”上某一点“生长”达到的。重复“生长”的过程，直到“树”的节点到达目标，或者到达可以直接到达目标节点的位置，算法完成，回溯节点得出路径。

假设我们要从A到达V，显然是不能直接直线到达的，使用RRT算法规划一条路径。首先将可到达格子（A - V）添加到备选节点列表，设置A为起始节点，V为目标节点。将A作为随机树的根节点，从备选列表中随机选取节点。假设第一次选取结果为O，显然O是可以直接与A连接的，将O从备选节点列表移除，并将父节点设置为A并将O加入到随机树，观察可得，O与V之间是有障碍物的，并不能直接到达V；



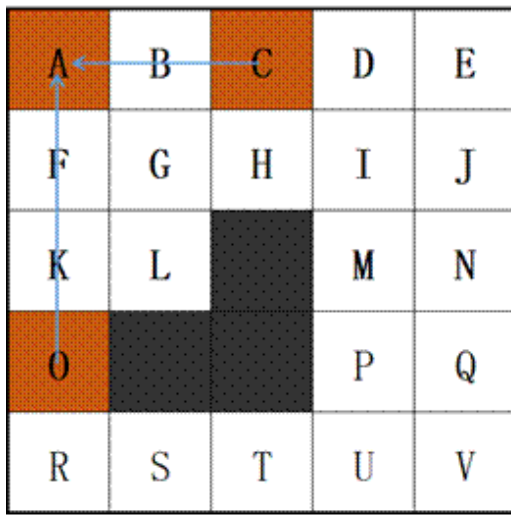
节点 O 加入随机树

然后我们可以继续从备选节点列表选取新的点，假设下一次获得的点是M，我们可以发现M距离树中的点O较近，但是M与O中间是有障碍物的，这时选择将M放回备选节点列表中，对随机树不做改动。

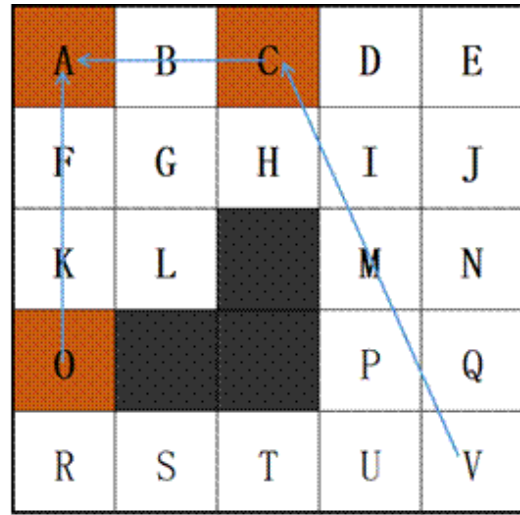
假设下次选取到节点C，我们对比树中记录的信息，可以发现C距离树中的节点A较近，并且C是可以直接到达A的，我们将C从备选节点列表移除，将父节点设置为A并加入到随机树中（图5）。此时我们尝试从C直接到达V，直观来看是没有障碍物的，但是计算机怎样识别有没有障碍物呢，这就需要我们设计一个函数来做判断。

首先，我们需要先获取C到V的欧氏距离，此处可以简单计算得到距离 $d_{CV} = \sqrt{2^2 + 4^2} = 2\sqrt{5} \approx 4$ ，我们可以认为C到V需要走四步，走完四步我们就可以到达V，那么C与V之间就有三个节点，如果这三个节点都不是障碍物，计算机就可以认为C是可以直接连通到V的。但是怎么确定这三个节点的位置呢？我们可以把C与V用直线段相连，并把该直线段均分为四等分，然后判断每一等分的中点在哪个栅格中，我们就认为通过了哪个节点，最后一段的中点就在目标节点V内。

我们可以发现节点C与目标节点V之间是没有障碍物的，可以认为C可以直接走到V，RRT算法得到了从A到达V的路径（图7）：A—>B，B—>C，C—>I，I—>M，M—Q，Q—>V。



节点 C 加入随机树



RRT 得出从 A 到 V 路径

由上例可发现，RRT规划路径的过程是比Dijkstra和AStar简单的，但是规划的结果并不比Dijkstra和AStar更优，在三维空间做路径规划时，使用Dijkstra和AStar必须要对运动空间进行建模（栅格图转化二维矩阵），这一步代价是比较大的，而使用RRT是可以不建模的，取随机点后我们只需要对树的拓展进行碰撞检测，就可以快速完成树的扩展找到可行的路径。

4.1.4 DWA算法

Fox 与 Thrun 等人在 1997 年提出了自动驾驶汽车局部避障的动态窗口法(Dynamic Window Approach, DWA)。动态窗口法 DWA 是在约束的速度矢量空间内采样线速度与角速度作为代价点，之后运用自动驾驶汽车运动学计算其待代价轨迹，之后对待代价轨迹进行评分选取最优轨迹，保证了自动驾驶汽车运动速度命令的可行性。

动态窗口法 DWA 的实现包含两个步骤：

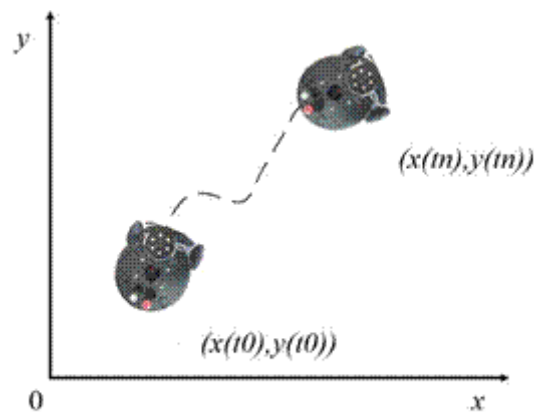
对自动驾驶汽车速度进行约束限制，形成动态窗口进行速度采样；

根据速度采样点求出待代价轨迹，最大化代价函数（或最小化代价函数）选取合适路径对应速度命令。

动态窗口是由一系列的约束构成，其中约束主要包括差速自动驾驶汽车的非完整约束、环境障碍物约束与受结构与电机影响的动力学约束。

自动驾驶汽车底盘的运动模型

下图所示为差速自动驾驶汽车运动学模型示意图， v_l 与 v_r 代表自动驾驶汽车底盘左右轮的线速度， $v(t)$ 和 $w(t)$ 分别代表自动驾驶汽车在世界坐标系下的平移速度与转动速度， L 代表自动驾驶汽车左右轮距。

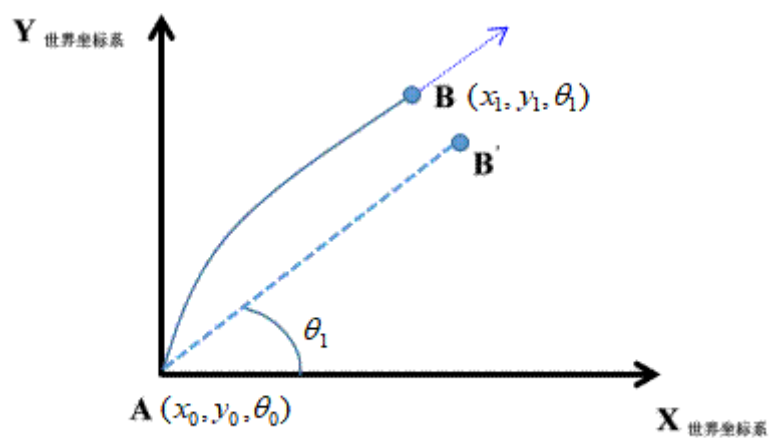


差速机器人运动学模型示意图

$$v(t) = \frac{(v_l(t) + v_r(t))}{2}$$

$$w(t) = \frac{(v_r(t) - v_l(t))}{L}$$

因为自动驾驶汽车在编码器采样周期内位移较小，所以在相邻两个时间采样点内将自动驾驶汽车运动轨迹近似为直线，即自动驾驶汽车沿着其航向角方向平移 $v(t)\Delta t$ 之后其航向角相对于世界坐标系转动 $w(t)\Delta t$ 。将该段距离与角度转换到世界坐标系下即为**自动驾驶汽车的轨迹**。则自动驾驶汽车运动轨迹可由如下简图与公式表示：



运动轨迹简化

A点设为当前点，B点为下一个轨迹点，通过A点的位姿信息与时间间隔，计算B点的位姿信息，我们采用以直代曲的思想，用B'，公式如下：

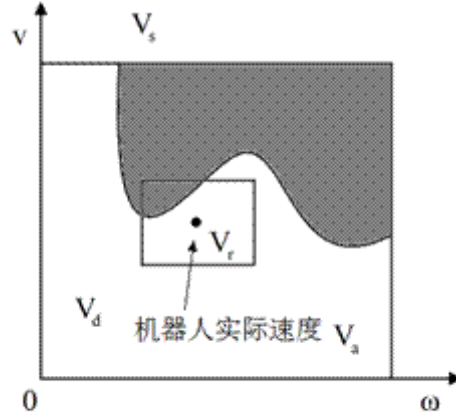
$$x_1 = x_0 + v \cos \theta_1 dt$$

$$y_1 = y_0 + v \sin \theta_1 dt$$

$$\theta_1 = \theta_0 + w dt$$

DWA算法原理与代价函数

下图为DWA算法的速度矢量空间示意图，图中横坐标为自动驾驶汽车角速度，纵坐标为自动驾驶汽车线速度，整个区域为 V_s ，所有白色区域 V_d 为避免碰撞的可行区域，灰色区域的速度表示执行该速度可能造成自动驾驶汽车碰撞障碍物， V_a 为考虑电机扭矩在控制周期内限制的自动驾驶汽车可达速度范围， V_r 为上述三个集合的交集，即最终确定的动态窗口。



速度矢量空间示意图

定义 V_s 为自动驾驶汽车线速度与角速度的集合即DWA算法搜索求解的最大范围，可用下式表示：

$$V_s = \{(v, w) | v \in [v_{\min}, v_{\max}] \wedge w \in [w_{\min}, w_{\max}]\}$$

其中

V_s ——能够达到的所有速度矢量的集合；

$v_{\max}, v_{\min}$ ——为自动驾驶汽车最大、最小线速度；

$w_{\max}, w_{\min}$ ——为自动驾驶汽车最大、最小角速度；

定义自动驾驶汽车无碰撞可行区域的线速度与角速度集合 V_d ，可表示如下：

$$V_d = \{(v, w) | v \leq \sqrt{2 \text{dist}(v, w) \dot{v}_s} \wedge w \leq \sqrt{2 \text{dist}(v, w) \dot{w}_s}\}$$

其中

$\dot{v}_s$ ——为自动驾驶汽车最大线减速度；

$\dot{w}_s$ ——为自动驾驶汽车最大角减速度；

$\text{dist}(v, w)$ ——为速度 (v, w) 对应轨迹上离障碍物最近的距离；

需要考虑由于电机转矩的限制，在控制周期内存在自动驾驶汽车最大、最小可达的速度变化范围，所以需要进一步缩小动态窗口，定义动态窗口 V_a 如下：

$$V_a = \{(v, w) | v \in [v_r - \dot{v}_s \Delta t, v_r + \dot{v}_s \Delta t] \wedge w \in [w_r - \dot{w}_s \Delta t, w_r + \dot{w}_s \Delta t]\}$$

其中

v ——为自动驾驶汽车一个采样周期内线速度范围；

w ——为自动驾驶汽车一个采样周期内角速度范围；

v_r ——为自动驾驶汽车现在的线速度；

ω_c ——为自动驾驶汽车现在的角速度；

$\dot{v}_a$ 、 $\dot{v}_b$ ——为自动驾驶汽车最大线加、减速度；

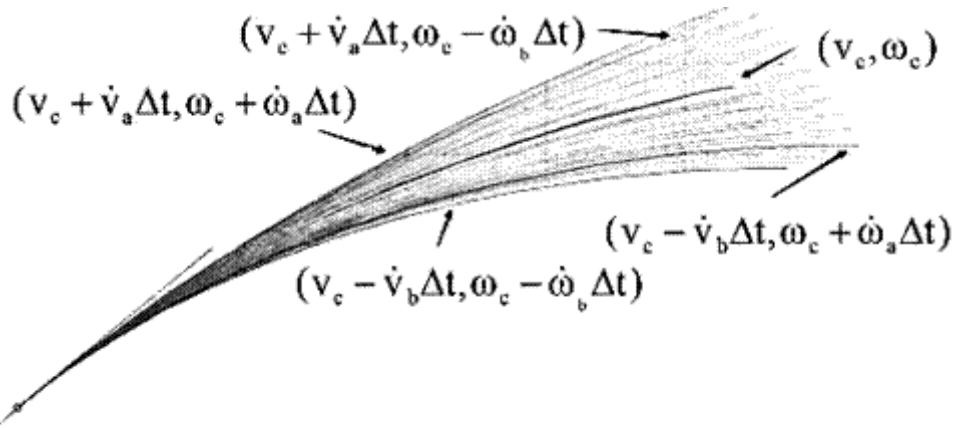
$\dot{\omega}_a$ 、 $\dot{\omega}_b$ ——为自动驾驶汽车最大角加、减速度；

Δt ——为自动驾驶汽车采样周期；

综合上述自动驾驶汽车最大速度约束、无碰撞约束与电机转矩约束，定义动态窗口 $\mathcal{V}_t$ 为：

$$\mathcal{V}_t = \mathcal{V}_s \cap \mathcal{V}_a \cap \mathcal{V}_\tau$$

在速度矢量空间 $\mathcal{V}_t$ 中，根据线速度、角速度采样点个数，将连续的速度矢量空间离散化，得到离散的采样点 $\{v, w\}$ 。对于每个采样点，根据自动驾驶汽车运动学模型模拟自动驾驶汽车前向仿真时间 t_{sim} 内自动驾驶汽车的运动轨迹，如图所示：



局部路径束

得到自动驾驶汽车运动轨迹之后，采用代价函数对这些轨迹进行评分，代价函数定义如下：

$$G(v, w) = \kappa(\alpha \text{Heading_diff}(v, w) + \beta \text{Goal_dist}(v, w) + \lambda \text{Path_dist}(v, w) + \sigma \text{Occ_cost}(v, w))$$

其中

κ 、 α 、 β 、 λ 、 σ 为各个代价函数的权重系数，决定对应代价重要性。

$\text{Heading_diff}(v, w)$ ——自动驾驶汽车方向角代价子函数；

$\text{Goal_dist}(v, w)$ ——自动驾驶汽车与终点欧氏距离代价子函数；

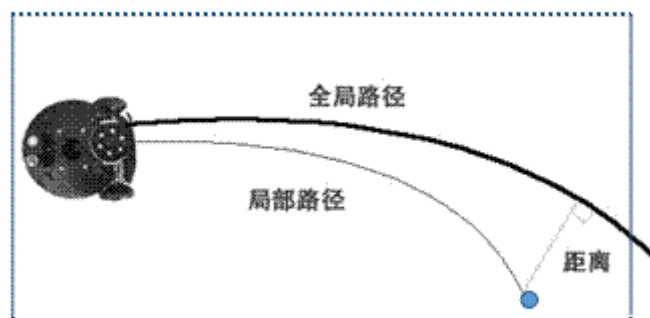
$\text{Path_dist}(v, w)$ ——全局规划路径代价子函数；

$\text{Occ_cost}(v, w)$ ——自动驾驶汽车到障碍物的距离代价子函数；

当 $G(v, w)$ 值为最小的时候，对应的便是最佳路径。

① 路径代价子函数

$\text{Path_dist}(v, w)$ 路径代价子函数计算局部轨迹末端点到全局路径的距离，距离越短说明轨迹越靠近全局路径。本实验中，全局路径使用了A-star算法求得。



路径代价示意图

$$Path_dist(v, w) = \min \sqrt{(x_{wp} - x_p)^2 + (y_{wp} - y_p)^2}$$

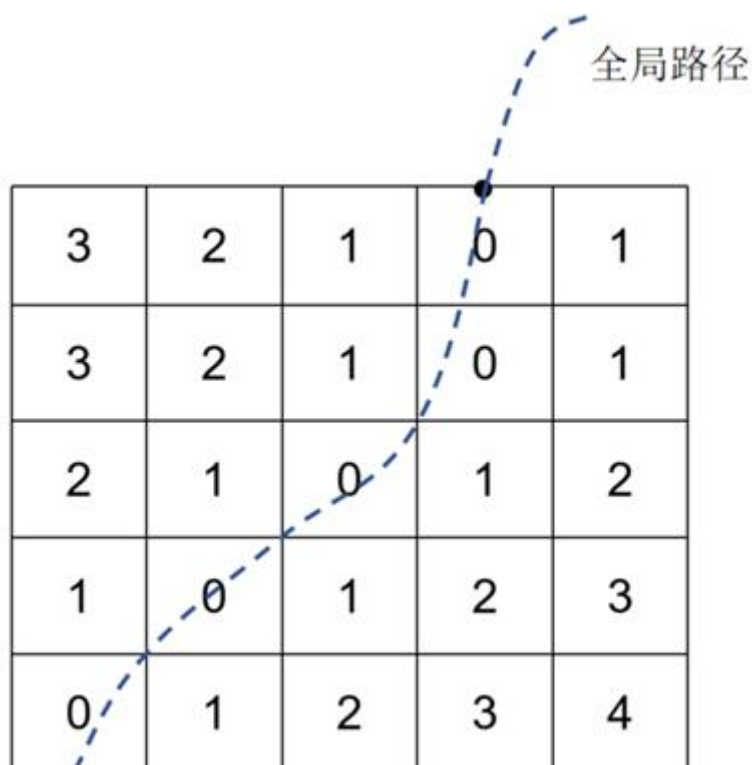
其中

(x_{wp}, y_{wp}) ——利用自动驾驶汽车运动学模型计算的局部路径末端点坐标；

(x_p, y_p) ——规划的全局路径离散点（本实验用的是A-star算法）；

下图更清晰的说明了路径代价的计算方式，此图为栅格地图，每个格子中的数字是对应的路径代价，我们知道局部路径越靠近全局路径越好，所以全局路径（虚线）经过的格子路径代价为0，越是远离全局路径代价的栅格，路径代价越大。

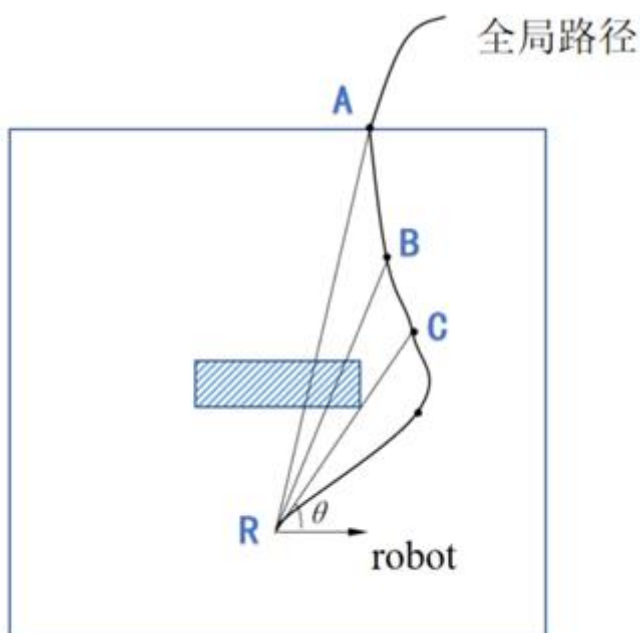
栅格地图路径代价示意图



② 自动驾驶汽车方向代价子函数

$Heading_dist(v, w)$ 是为了使自动驾驶汽车在移动过程中，其方位角不断朝向终点位置，如下图：

方向代价示意图



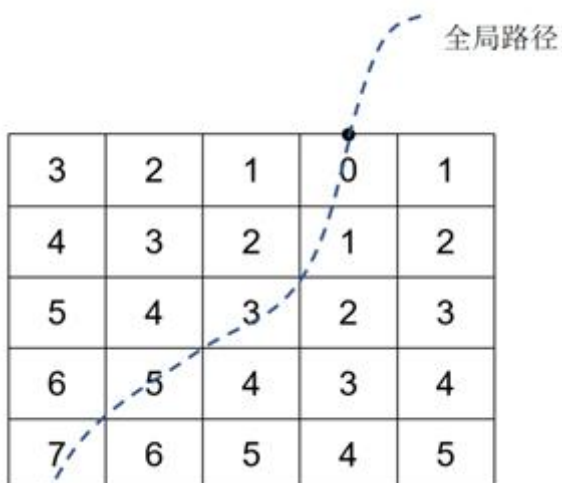
方向代价的确定方法是首先是找到一个基准点，我们先找到动态窗口与全局路径的交点如图A，连接局部路径终点R，如果连线穿过障碍物，如A点与B点，此时这两个点就不能作为基准点，直到寻找第一个连线不通过障碍物的点，如点C，直线CR与局部路径自动驾驶汽车方向形成的夹角 θ ，便是方向代价。不难发现自动驾驶汽车越往A点偏，方向代价越小。

计算方向代价的方法有很多，本实验的方向代价计算方法会在本文最后进行详细讲解。

③ 自动驾驶汽车与终点距离代价子函数

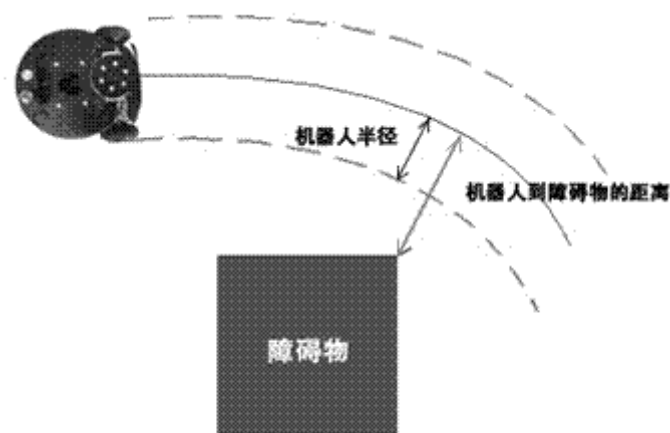
$Goal_dist(v, w)$ 是自动驾驶汽车局部路径末端点到局部路径窗口与全局路径交点的距离，其目的是不断缩短与终点的距离。

栅格地图终点代价示意图



④ 自动驾驶汽车到障碍物距离代价子函数

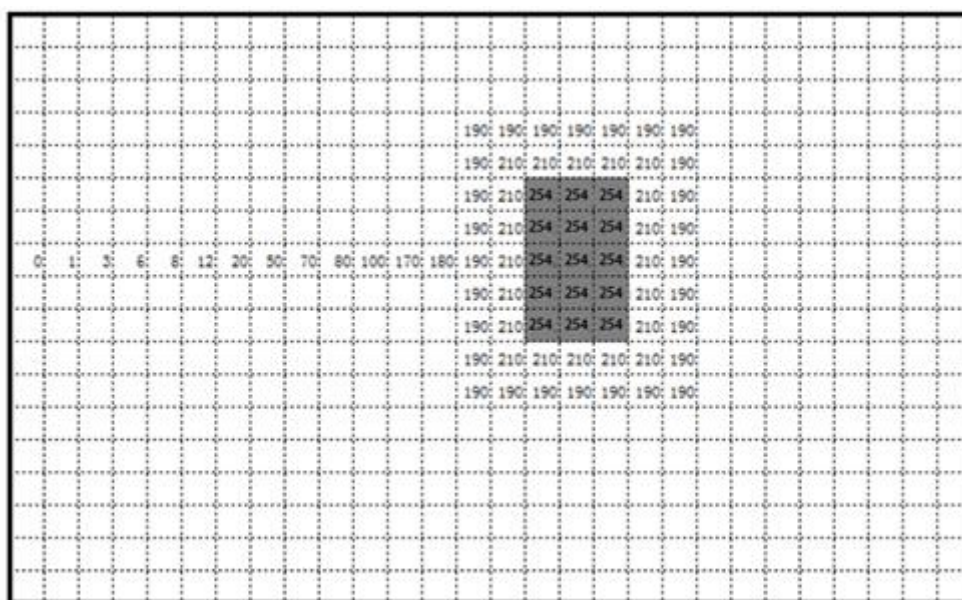
$Occ_cost(x, y)$ 用于代价自动驾驶汽车轨迹到障碍物的距离，体现了自动驾驶汽车的避障能力。如果自动驾驶汽车的轨迹到障碍物的距离大于自动驾驶汽车的半径，则没有发生碰撞的危险，我们可以设置评分为0；反之，就说明碰撞风险较大，设置评分为-1，抛弃这条轨迹。



障碍物代价示意图

下图是栅格地图的障碍物代价情况，可以看出在障碍物中的栅格障碍物代价是最大值254，距离障碍物越远，障碍物代价越小。

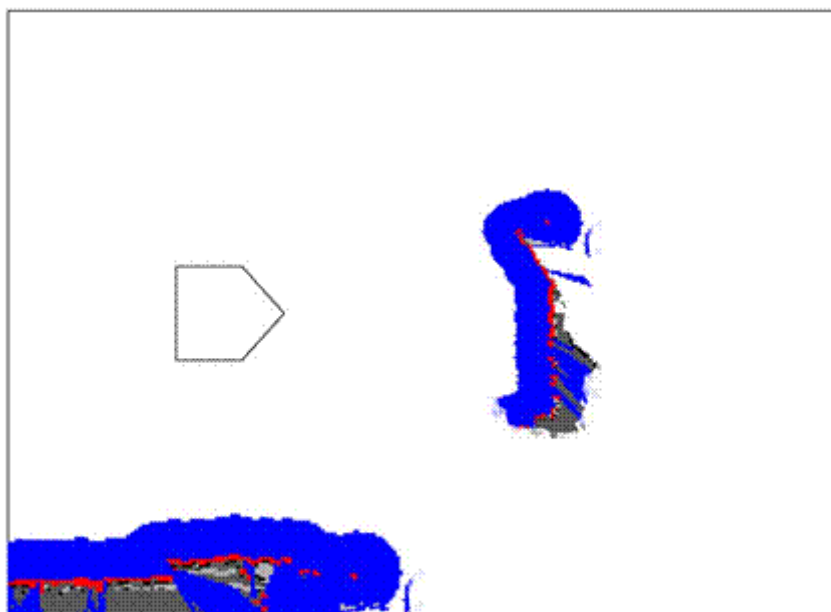
栅格地图障碍物代价示意图



上述便是DWA算法的原理，到这里我们初步了解了如何利用DWA算法原理完成局部路径规划避障功能，接下来我们结合代码，看一看具体实现与原理之间的差异。

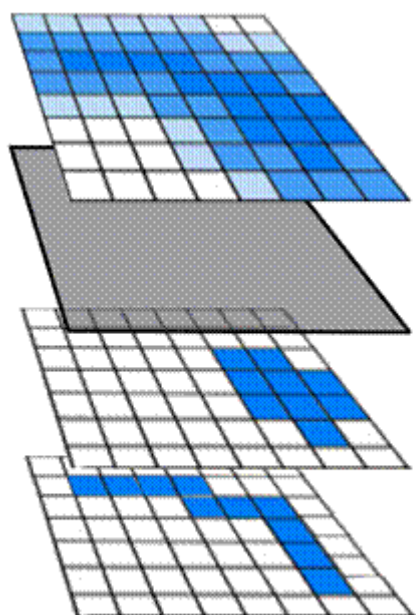
知识补充：ROS中的costmap

实验中的costmap是通过自动驾驶汽车传感器的信息，本实验为激光雷达信息，建立和更新二维地图。下图中，红色的点表示障碍物，蓝色部分表示通过自动驾驶汽车底盘内切圆半径作为膨胀依据的障碍物情况，灰色的五边形为自动驾驶汽车轮廓垂直投影（footprint）：



Costmap 示意图

costmap是由多个层叠加得到，第一层为静态地图层StaticLayer，此地图层表示的是已知固定障碍物的地图环境，第二层为突发障碍物层ObstacleLayer，第三层为膨胀操作层InflationLayer，此层会将StaticLayer中的障碍物以及ObstacleLayer层的突发障碍物进行膨胀最后得到costmap，示意图如下：



costmap 各层示意图

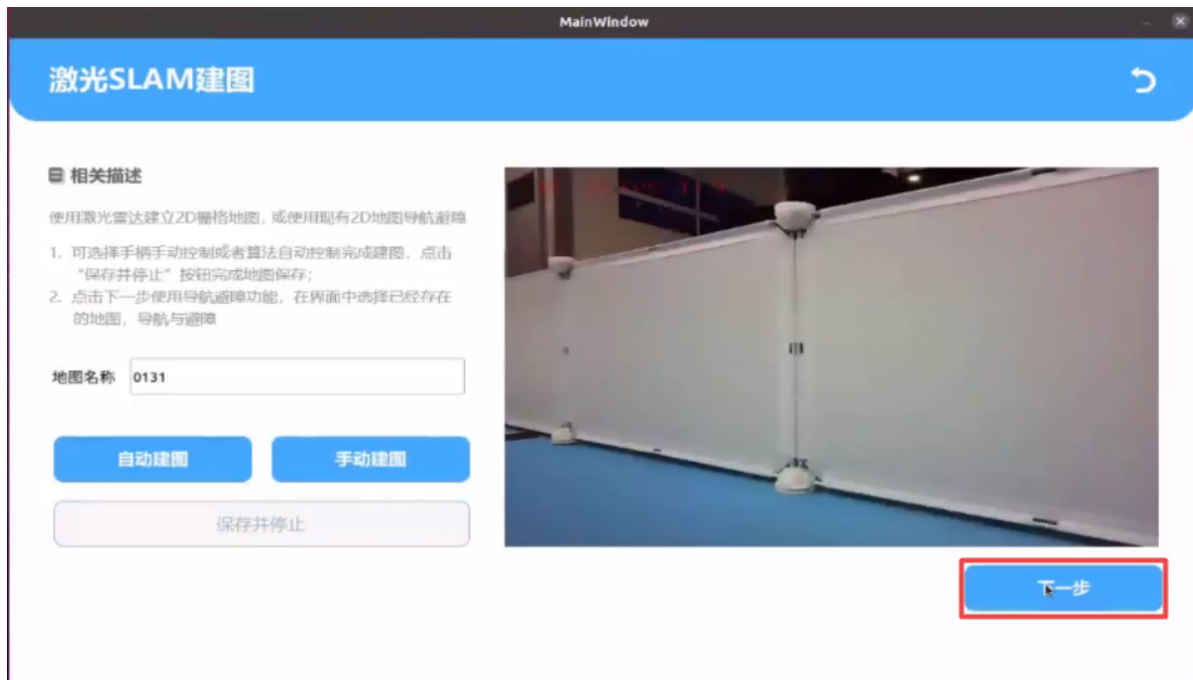
五、任务步骤

步骤一 建图

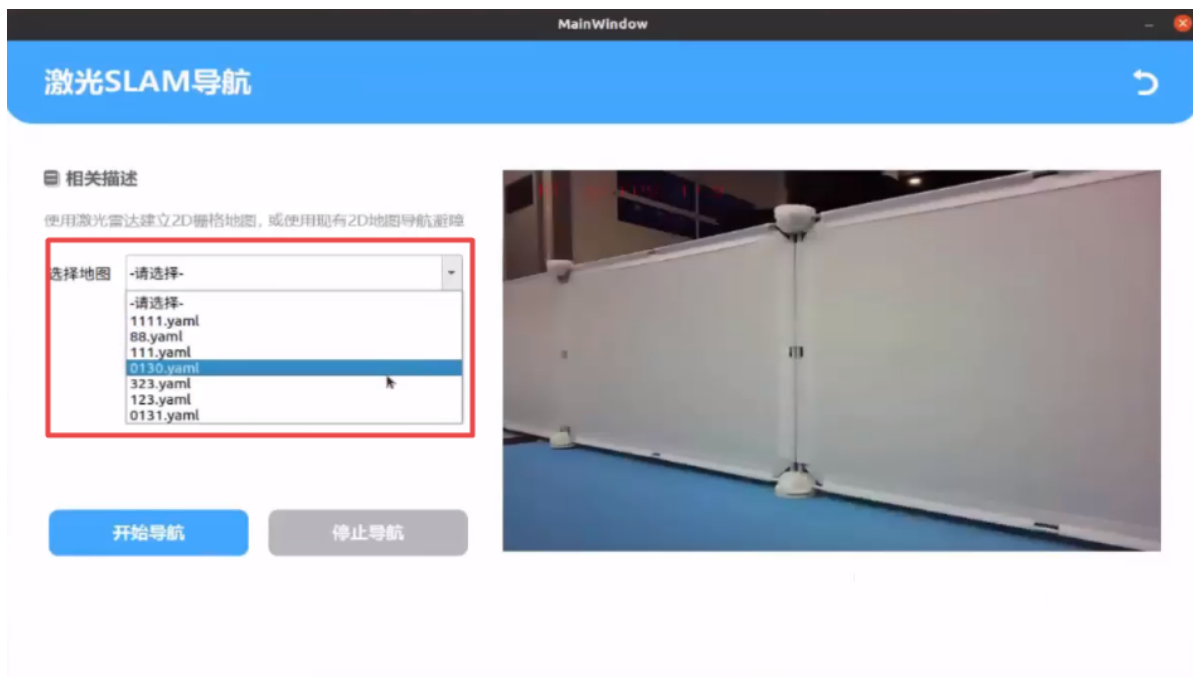
本任务请参照SLAM建图，自动或者手动建图，将地图保存。

步骤二 开启导航

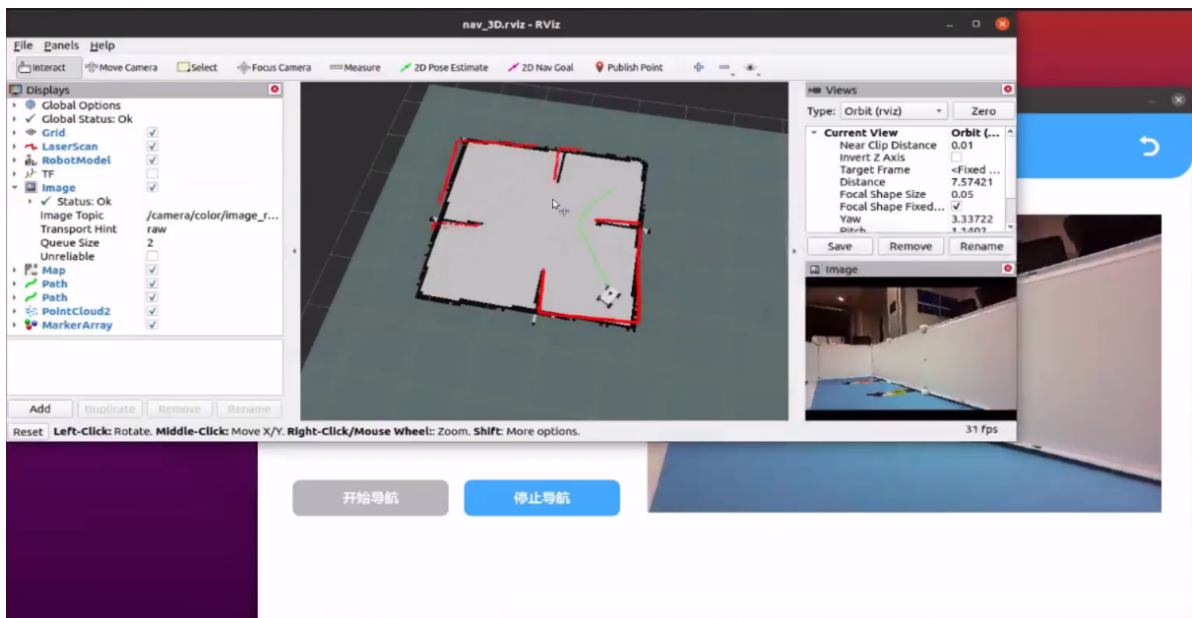
建图完成后，点击“下一步”，进入导航页面：



选择创建好的地图：



点击“开始导航”，开启RVIZ，进入自动导航页面：



小车在真实地图中按照图上的路线行进：



步骤三 代码流程实现

1. 创建文件"/home/mo/ros_space/src/ai_node_master/launch/navigation_2d.launch"，添加如下内容，实现自动驾驶汽车实现SLAM激光雷达导航的主要流程，本此实训我们关注的是避障节点和自动导航节点。

```
<!--param参数配置-->
<launch>
  <param name="node_id_closer" value="/navigation" />
  <arg name="map" default="mymap2.yaml" />
  <arg name="rotate_speed" default="0.22" />
  <arg name="rotation_flg" default="false" />
  <arg name="nav_sim_flg" default="false" />
  <arg name="3d_enable" default="false" />
  <arg name="mapping" default="false" />
  <node pkg="ai_node_master" type="ros_closer" name="navigation"
  output="screen" required="true" />

  <!-- 激光雷达驱动 -->
  <include file="$(find rplidar_ros)/launch/rplidar.launch" />
</launch>
```

```

<!-- 底盘与深度摄像头API -->
<include file="$(find ai_node_master)/cfg/API_cfg.launch" >
</include>

<!-- 运行地图服务器，并且加载设置的地图-->
<node name="map_server" pkg="map_server" type="map_server" args="$(find
ai_node_master)/maps/$(arg map)"/>

<!-- 避障节点 -->
<include file="$(find navigation_settings)launch/move_base.launch.xml" >
    <arg name="3d_enable"          default="$(arg 3d_enable)" />
    <arg name="mapping"            default="$(arg mapping)" />
</include>

<!-- 启动AMCL节点 -->
<arg name="custom_amcl_launch_file" default="$(find
navigation_settings)launch/robot_amcl.launch.xml"/>
<include file="$(arg custom_amcl_launch_file)"></include>

<!-- 自动导航节点 -->
<include file="$(find loop_nav)/launch/loop_nav_3d.launch" >
    <arg name="rotate_speed"      default="$(arg rotate_speed)" />
    <arg name="rotation_flage"    default="$(arg rotation_flage)" />
    <arg name="nav_sim_flag"      default="$(arg nav_sim_flag)" />
</include>

<!-- 手动设定多点导航节点 -->
<!-- <node pkg="loop_nav" type="send_mark_reverse.py" name="send_mark"
output="screen">
    <param name="reverse_flag" value="true"/>
</node> -->

<!-- 可视化界面 -->
<node pkg="rviz" type="rviz" name="rviz" args="-d $(find
urdf_car)/rviz/nav_3D.rviz" output="screen"/>

</launch>

```

2. 编写避障节点"/home/mo/ros_space/src/navigation_settings/launch/move_base.launch.xml"，添加如下内容，实现不同场景下的不同避障功能：

```

<!--
    ROS navigation stack with velocity smoother and safety (reactive)
    controller
-->
<launch>

    <arg name="odom_frame_id"      default="odom"/>
    <arg name="base_frame_id"      default="base_footprint"/>
    <arg name="global_frame_id"    default="map"/>
    <arg name="odom_topic"         default="odom" />
    <arg name="laser_topic"        default="scan" />
    <arg name="3d_enable"          default="true" />
    <arg name="mapping"            default="true" />

```

```

    <node pkg="move_base" type="move_base" respawn="false" name="move_base"
output="screen">

<param name="controller_frequency" value="5"/>
<param name="clearing_rotation_allowed" value="false"/>

<!-- D* -->
<!-- <param name="base_global_planner"
value="dstar_global_planner/DStarPlannerROS"/> -->

<!-- A* & Dijkstra -->
<param name="base_global_planner" value="global_planner/GlobalPlanner"/>

<!-- default -->
<!-- <param name="base_global_planner" value="navfn/NavfnROS"/> -->
<!-- <param name="base_local_planner"
value="base_local_planner/TrajectoryPlannerROS"/> -->

<rosparam unless="$(arg 3d_enable)" file="$(find
navigation_settings)/param/costmap_common_params.yaml" command="load"
ns="global_costmap" />
<rosparam unless="$(arg 3d_enable)" file="$(find
navigation_settings)/param/costmap_common_params.yaml" command="load"
ns="local_costmap" />

<rosparam if="$(arg 3d_enable)" file="$(find
navigation_settings)/param/costmap_common_params_3D.yaml" command="load"
ns="global_costmap" />
<rosparam if="$(arg 3d_enable)" file="$(find
navigation_settings)/param/costmap_common_params_3D.yaml" command="load"
ns="local_costmap" />

<rosparam if="$(arg mapping)" file="$(find
navigation_settings)/param/dwa_local_planner_params_mapping.yaml"
command="load" />
<rosparam unless="$(arg mapping)" file="$(find
navigation_settings)/param/dwa_local_planner_params.yaml" command="load" />

<rosparam file="$(find navigation_settings)/param/local_costmap_params.yaml"
command="load" />
<rosparam file="$(find
navigation_settings)/param/global_costmap_params.yaml" command="load" />
<!-- <rosparam file="$(find
navigation_settings)/param/base_local_planner_params.yaml" command="load" />
-->
<rosparam file="$(find navigation_settings)/param/move_base_params.yaml"
command="load" />
<rosparam file="$(find
navigation_settings)/param/global_planner_params.yaml" command="load" />
<rosparam file="$(find
navigation_settings)/param/navfn_global_planner_params.yaml" command="load"
/>

<!-- reset frame_id parameters using user input data -->
<param name="global_costmap/global_frame" value="$(arg global_frame_id)"/>
<param name="global_costmap/robot_base_frame" value="$(arg base_frame_id)"/>
<param name="local_costmap/global_frame" value="$(arg odom_frame_id)"/>

```

```

<param name="local_costmap/robot_base_frame" value="$(arg base_frame_id)"/>
<param name="DWAPlannerROS/global_frame_id" value="$(arg odom_frame_id)"/>

<remap from="odom" to="$(arg odom_topic)"/>
<remap from="scan" to="$(arg laser_topic)"/>
<remap from="cmd_vel" to="/cmd_vel"/>

  </node>
</launch>

```

以下代码指定dwa算法所用到的参数配置：

```

<rosparam if="$(arg mapping)" file="$(find
navigation_settings)/param/dwa_local_planner_params_mapping.yaml"
command="load" />

```

配置文

件"/home/mo/ros_space/src/navigation_settings/param/dwa_local_planner_params_mapping.yaml"具体内容如下：

```

DWAPlannerROS:

  # use_dwa: true

# Robot Configuration Parameters

  max_vel_x: 0.11
  min_vel_x: -0.03

  max_vel_y: 0.1
  min_vel_y: -0.1

  # The velocity when robot is moving in a straight line

  max_vel_trans: 0.12
  min_vel_trans: 0.015
  trans_stopped_vel: 0.01

  max_vel_theta: 0.5
  min_vel_theta: 0.2
  theta_stopped_vel: 0.05

  # escape_vel: -0.1

  # min_in_place_vel_theta: 0.1

  acc_lim_x: 3
  acc_lim_theta: 3
  acc_lim_y: 3

# Goal Tolerance Parametes

  yaw_goal_tolerance: 0.15
  xy_goal_tolerance: 0.08
  latch_xy_goal_tolerance: false

```



```

# Forward Simulation Parameters

sim_time: 2
vx_samples: 5
vy_samples: 7
vtheta_samples: 6
controller_frequency : 20

# Trajectory Scoring Parameters

path_distance_bias: 32.0
goal_distance_bias: 24.0
occdist_scale: 0.02
#twirling_scale: 0
forward_point_distance: 0.325
stop_time_buffer: 0.4
scaling_speed: 0.25
max_scaling_factor: 0.2

# Oscillation Prevention Parameters

oscillation_reset_dist: 0.05

# Debugging

publish_traj_pc : true
publish_cost_grid_pc: true
global_frame_id: map

```

以下代码指定全局路径规划算法所用到的参数配置：

```

<rosparam file="$(find
navigation_settings)/param/global_planner_params.yaml" command="load" />

```

配置文

件"/home/mo/ros_space/src/navigation_settings/param/global_planner_params.yaml"具体内容如下：

```

GlobalPlanner:                                     # Also see:
http://wiki.ros.org/global_planner
  old_navfn_behavior: false                         # Exactly mirror behavior of
navfn, use defaults for other boolean parameters, default false
  use_quadratic: true                               # Use the quadratic
approximation of the potential. Otherwise, use a simpler calculation,
default true
  use_dijkstra: true                               # Use dijkstra's algorithm.
Otherwise, A*, default true
  use_grid_path: false                             # Create a path that follows
the grid boundaries. Otherwise, use a gradient descent method, default false

  allow_unknown: true                              # Allow planner to plan
through unknown space, default true

```



```

#Needs to have
track_unknown_space: true in the obstacle / voxel layer (in
costmap_commons_param) to work
    planner_window_x: 0.0                # default 0.0
    planner_window_y: 0.0                # default 0.0
    default_tolerance: 0.0               # If goal in obstacle, plan
to the closest point in radius default_tolerance, default 0.0

    publish_scale: 100                   # Scale by which the
published potential gets multiplied, default 100
    planner_costmap_publish_frequency: 0.0 # default 0.0

    lethal_cost: 253                     # default 253
    neutral_cost: 50                     # default 50
    cost_factor: 3                       # Factor to multiply each cost
from costmap by, default 3.0
    publish_potential: true               # Publish Potential Costmap
(this is not like the navfn pointcloud2 potential), default true

```

同学们可以试着修改以上配置文件的参数，来观察自动驾驶汽车在行进过程中对于障碍物的躲避效果。

3. 编写自动驾驶节点“/home/mo/ros_space/src/loop_nav/launch/loop_nav_3d.launch”，添加如下内容：

```

<launch>

<arg name="rotate_speed"          default="0.22" />
<arg name="rotation_flag"         default="true" />
<arg name="nav_sim_flag"          default="false" />

<rosparam unless="$(arg nav_sim_flag)"
file="/home/mo/ros_space/map_size.yaml" command="load" />
<rosparam if="$(arg nav_sim_flag)">
    map_x : 4.5
    map_y : 3
</rosparam>

<node pkg="loop_nav" type="explore_point_3d" name="explore_point_3d"
respawn="true" output="screen">
    <param name="rotate_speed"      value="$(arg rotate_speed)"/>
    <param name="rotation_flag"     value="$(arg rotation_flag)"/>
</node>

</launch>

```

explore_point_3d节点实现导航功能，新

建“/home/mo/ros_space/src/loop_nav/src/explore_point_3d.cpp”文件，添加如下内容：

```

#include <ros/ros.h>
#include <move_base_msgs/MoveBaseAction.h>
#include <geometry_msgs/Twist.h>
#include <actionlib/client/simple_action_client.h>
#include <boost/thread.hpp>
#include <vector>

```

```

typedef actionlib::SimpleActionClient<move_base_msgs::MoveBaseAction>
    MoveBaseClient;
float rotateTime;
float map_x, map_y = 0;
bool rotation_flg;
move_base_msgs::MoveBaseGoal goal;
geometry_msgs::Twist twist;
std::vector<move_base_msgs::MoveBaseGoal> goal_pool;
int count = 0;
int goal_status;

// 订阅 move_base/feedback 话题，有变化后调用 goalManager，处理 feedback 并计算
// process。

void changeGoal( MoveBaseClient &ac_){
    ROS_INFO("Change to the next goal!");

    goal = goal_pool[count];
    goal.target_pose.header.stamp = ros::Time::now();
    ac_.sendGoal(goal);

    count += 1;
    if(count == goal_pool.size())
        count = 0;
}

void fillGoalPool(){
    // ROS_INFO("    %i" ,goalcount);

    // ROS_INFO("    %f" ,goal_pool[i].target_pose.pose.position.x);
    // if (goal_.goal.target_pose.pose.position.x ==
    goal_pool[i].target_pose.pose.position.x &&
    //     goal_.goal.target_pose.pose.position.y ==
    goal_pool[i].target_pose.pose.position.y)
    // {
    //     return;
    // }

    goal.target_pose.header.frame_id = "map";

    goal.target_pose.pose.position.x = map_x - 1.1;
    goal.target_pose.pose.position.y = 0.2;
    goal.target_pose.pose.orientation.w = 0;
    goal.target_pose.pose.orientation.z = 1;

    goal_pool.push_back(goal) ;

    goal.target_pose.pose.position.x = map_x - 1.1;
    goal.target_pose.pose.position.y = map_y - 1.1;

    goal_pool.push_back(goal) ;

    goal.target_pose.pose.position.x = 0.2;
    goal.target_pose.pose.position.y = map_y - 0.95;
    goal.target_pose.pose.orientation.w = 1;

```

```

goal.target_pose.pose.orientation.z = 0;
goal_pool.push_back(goal) ;

goal.target_pose.pose.position.x = 0;
goal.target_pose.pose.position.y = 0;

goal_pool.push_back(goal) ;

// ROS_INFO("    %i" ,goal_pool.size());

}

void resultLisenner( const actionlib_msgs::GoalStatusArray &status){
    goal_status = int(status.status_list[0].status) *
status.status_list.size();
}

int main(int argc, char** argv) {
    ros::init(argc, argv, "send_goals_node");
    ros::NodeHandle node;

    ros::Publisher vel_pub = node.advertise<geometry_msgs::Twist>("/cmd_vel",
10);
    ros::Rate loop_rate(2);
    ros::param::param<float>("map_x", map_x, 3);
    ros::param::param<float>("map_y", map_y, 3);
    ros::param::param<float>("~rotate_speed", rotateTime, 0.22);
    ros::param::param<bool> ("~rotation_flage", rotation_flage, false);
    ros::Duration(4).sleep();
    MoveBaseClient ac("move_base", true);
    ac.waitForServer();

    fillGoalPool();
    twist.angular.z = -rotateTime;

    changeGoal(ac);

    ROS_INFO("star nav");
    while (ros::ok()) {
        loop_rate.sleep();
        auto state = ac.getState();
        ros::Subscriber sub = node.subscribe("/move_base/status", 1,
resultLisenner);
        if ( state == actionlib::SimpleClientGoalState::SUCCEEDED)
        {
            ROS_INFO("Goal reached! Change to the next goal in");
            if (rotation_flage){
                double i = ros::Time().now().toSec();
                while (ros::Time().now().toSec() - i < 6.3/rotateTime )
                {
                    vel_pub.publish(twist);
                    loop_rate.sleep();
                }
            }
            changeGoal(ac);
        }
    }
}

```

```

    }
    else if(state == actionlib::SimpleClientGoalState::PENDING){
        // ROS_INFO("Planning");
    }
    else if(state == actionlib::SimpleClientGoalState::ACTIVE){
        // ROS_INFO("Moving");
    }
    else if(state == actionlib::SimpleClientGoalState::PREEMPTED){
        ros::Duration(1).sleep();
        if (goal_status == 3)
        {
            changeGoal(ac);
            goal_status = 0;
        }
    }
    else
    {
        ROS_INFO("Fail to reach the goal!Change to the next goal in");
        if (rotation_flag){
            double i = ros::Time().now().toSec();
            while (ros::Time().now().toSec() - i < 6.3/rotateTime )
            {
                vel_pub.publish(twist);
                loop_rate.sleep();
            }
        }
        changeGoal(ac);
    }

    ros::spinOnce();
}
}

```

六、任务总结

全局路径规划是依据给定的目标，接受权值地图信息生成全局权值地图，规划出从起点到目标位置的全局路径，作为局部路径规划的参考。本文介绍的Dijkstra算法，A*算法和RRT算法，都属于全局路径规划算法。

局部路径规划层是作为导航系统的局部规划部分，接受权值地图生成的局部权值地图信息，依据附近的障碍物信息进行局部路径规划。本文介绍的DWA算法属于局部路径规划算法。